Case Study: Database Architecture & Data Storage Strategy at Netflix

---

1. Introduction & System Scale

Netflix is a global streaming giant serving hundreds of millions of active accounts worldwide. To deliver seamless video playback, real-time recommendations, and continuous user tracking under massive concurrent loads, a traditional single-database approach is impossible.

---

2. Relational vs. Non-Relational Storage at Netflix

- **Relational (SQL):** Constrained, hard-coded schemas, strict ACID compliance.
- **Non-Relational (NoSQL):** Flexible, unconstrained schemas, optimized for real-time scale and high-velocity reads/writes.

Where Netflix Uses Relational Databases (RDBMS)

Netflix relies on relational databases (such as **MySQL** and **PostgreSQL**) strictly for data that requires absolute structural integrity and transactional consistency:

- **Billing and Subscriptions:** Managing user memberships, credit card transactions, invoicing history, and financial ledgers.
- **Account Management:** Core user credential mappings where relational constraints prevent orphan records or inconsistent user profiles.

Where Netflix Uses Non-Relational Databases (NoSQL)

For the vast majority of its core streaming application features, Netflix leverages **NoSQL distributed databases** (such as Apache Cassandra, Amazon DynamoDB, and EVCache/Redis):

- **Viewing History & Resume Playback:** Tracking millions of concurrent users stopping and starting videos second-by-second requires write-heavy, highly scalable storage that a rigid relational table would struggle to process efficiently under high velocity.
- **Personalized Recommendation Engine:** Storing dynamic user metadata, viewing preferences, and real-time interaction logs that change rapidly and lack a fixed schema.

---

3. The Database Dilemma: Why Netflix Avoids Purely Relational Storage for Streaming

If Netflix attempted to store all streaming telemetry, user logs, and real-time recommendation states in a monolithic relational database, it would hit severe bottlenecks:

1. **Rigid Schema Constraints:** As user personalization metrics evolve, altering tables across petabytes of data causes massive downtime. NoSQL provides the schema flexibility required.
2. **Write Bottlenecks:** Relational systems enforce strict locking mechanisms to maintain consistency across tables. During peak evening streaming hours, millions of concurrent writes (e.g., updating timestamps for "Continue Watching") would choke an RDBMS.
3. **Horizontal Scalability:** NoSQL databases are purpose-built to scale out horizontally across distributed clusters, matching Netflix’s global cloud infrastructure (running primarily on AWS).

---

4. ACID Properties in the Netflix Ecosystem

While NoSQL handles high-speed streaming metadata, core components of Netflix still demand strict **ACID properties** (Atomicity, Consistency, Isolation, Durability) as taught in class:

- **Atomicity (All-or-Nothing Rule):** When a user updates their subscription plan or processes a monthly payment, the transaction either completely succeeds or rolls back entirely. Partial updates are unacceptable.
- **Consistency:** Ensures that a user's account status (e.g., active vs. canceled subscription) is instantly reflected across billing and access control layers.

- **Isolation:** Ensures that simultaneous updates from multiple devices on the same account (e.g., changing a password while another profile streams) do not corrupt transactional states.
- **Durability:** Once a payment or account modification is committed, it survives system crashes or server failures.

---

5. Conclusion: The Power of Multiple Databases

Netflix proves that modern enterprise architecture rarely relies on a "one-size-fits-all" database. By combining **relational databases** for strict financial and account transactions with **NoSQL systems** for flexible, real-time streaming data, Netflix achieves optimal performance, high availability, and global scale.
